Logiciel Bibliothèque QT : compte rendu

TABLES DES MATIÈRES :

Début du code : Base de données : 1

TABLE LIVRE : 2

TABLE CATEGORIE : 10

TABLE AUTEUR : 12

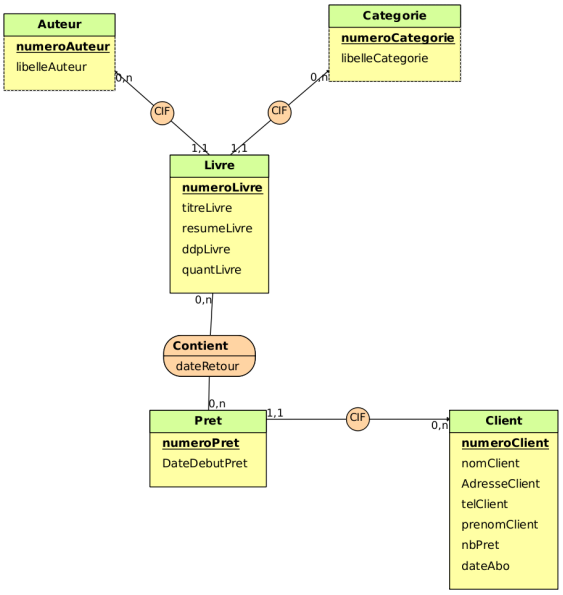
TABLEAU PRET : 14

Début du code : Base de données :

Mise en relation avec la base de données et header du code.

```
MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    QSqlDatabase dbBiblio = QSqlDatabase::addDatabase("QMYSQL");//Instanciation de la variable membre dbBiblio

    // Définition des paramètres de connexion à la base de données
    dbBiblio.setHostName("127.0.0.1"); // ip serveur MySQL
    dbBiblio.setDatabaseName("dbBiblio"); // Nom de la base
    dbBiblio.setUserName("ubittoni"); // Nom utilisateur
    dbBiblio.setPassword("elini01"); // Mot de passe

    if(dbBiblio.open()) {
    qDebug() << "Ok - ouverture de la base de donnée";
    }
    else {
    qDebug() << "Echec d'ouverture de la base de donnée";
    qDebug() << dbBiblio.lastError();
    }
```

Model conceptuel de donnée de la base :



Son code Sql :

CREATE TABLE `Auteur`(`numeroAuteur` INTEGER,`libelleAuteur` VARCHAR(25),primary key(`numeroAuteur`));

CREATE TABLE `Categorie`(`numeroCategorie` INTEGER,`libelleCategorie` VARCHAR(50),primary key(`numeroCategorie`));

CREATE TABLE `Livre`(`numeroLivre` INTEGER,`titreLivre` VARCHAR(50),`resumeLivre` VARCHAR(300),`ddpLivre` INTEGER,`quantLivre` INTEGER,`numeroAuteur` INTEGER NOT NULL,`numeroCategorie` INTEGER NOT NULL, foreign key (`numeroAuteur`) references Auteur(`numeroAuteur`), foreign key (`numeroCategorie`) references Categorie(`numeroCategorie`),primary key(`numeroLivre`));

CREATE TABLE `Client`(`numeroClient` INTEGER,`nomClient` VARCHAR(25),`AdresseClient` VARCHAR(50),`telClient` VARCHAR(10),`prenomClient` VARCHAR(25),`nbPret` VARCHAR(50),`dateAbo` VARCHAR(50),primary key(`numeroClient`));

CREATE TABLE `Pret`(`numeroPret` INTEGER,`DateDebutPret` DATE,`numeroClient` INTEGER NOT NULL, foreign key (`numeroClient`) references Client(`numeroClient`),primary key(`numeroPret`));

CREATE TABLE `Contient`(`dateRetour` DATE,`numeroLivre` INTEGER NOT NULL,`numeroPret` INTEGER NOT NULL, foreign key (`numeroLivre`) references Livre(`numeroLivre`), foreign key (`numeroPret`) references Pret(`numeroPret`),primary key(`numeroLivre`,`numeroPret`));

FICHIER .h :

Voici l'extrait de mon fichier.h :

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H
#include <QMainWindow>
#include <QtSql>
#include <QSqlDatabase>
#include <QMainWindow>
#include <QComboBox>
#include "dialogauthor.h"
#include "dialogcategorie.h"
#include <QDialog>

namespace Ui {
class MainWindow;
}

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = 0);
    ~MainWindow();

private slots:
    void on_pushButton_Ajouter_clicked();

    void on_pushButton_Modifier_clicked();

    void on_pushButton_Supprimer_clicked();

    void on_tableWidget_itemSelectionChanged();

    void on_actionQuit_triggered();

    void on_pushButton_Ajouter_Cat_clicked();

    void on_pushButton_Modifier_Cat_clicked();

    void on_pushButton_Suppr_Cat_clicked();

    void on_tableWidget_Cat_itemSelectionChanged();

    void on_pushButton_Ajouter_Aut_clicked();

    void on_pushButton_Modifier_Aut_clicked();

    void on_tableWidget_Aut_itemSelectionChanged();

    void on_pushButton_Suppr_Aut_clicked();

    void on_pushButton_Auteur_clicked();

    void on_pushButton_Cat_clicked();

    void on_lineEditRechercheAdherentPret_textEdited(const QString &arg1);

    void on_lineEditRechercherLivrePret_textEdited(const QString &arg1);

    void on_pushButtonAjouterLivrePret_clicked();

    void on_pushButton_Supprimer_Livre_clicked();

    void on_pushButtonEnregistrerPret_clicked();

    void on_listWidget_Adherent_Pret_itemSelectionChanged();

private:
    Ui::MainWindow *ui;
    QSqlDatabase dbBiblio;
    void afficherLivre();
    void afficherCategorie();
    void afficherAuteur();
    void remplirComboBox();
    int noLigne;
    QComboBox ComboBox;
};

#endif // MAINWINDOW_H
```

Celui-ci est long, avec essentiellement pour cause les multiples fonctions

TABLE LIVRE :

Création du tableau Livre :

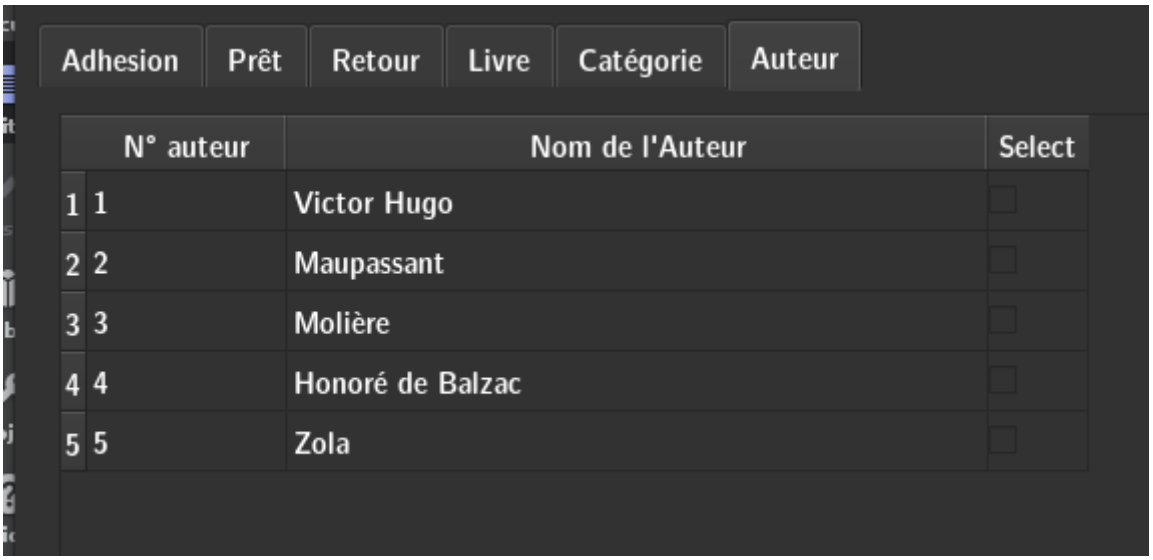
```
/************************TABLEAU LIVRE*************************/

//Création des lignes et colonnes
ui->tableWidget->setColumnCount(7);//Ajoute 6 Colonne

/***************JE LES NOMMES***************/
ui->tableWidget->setHorizontalHeaderItem(0,new QTableWidgetItem("N°"));
ui->tableWidget->setHorizontalHeaderItem(1,new QTableWidgetItem("Select"));
ui->tableWidget->setHorizontalHeaderItem(2,new QTableWidgetItem("Titre"));
ui->tableWidget->setHorizontalHeaderItem(3,new QTableWidgetItem("Date de publication"));
ui->tableWidget->setHorizontalHeaderItem(4,new QTableWidgetItem("Auteur"));
ui->tableWidget->setHorizontalHeaderItem(5,new QTableWidgetItem("Catégorie"));
ui->tableWidget->setHorizontalHeaderItem(6,new QTableWidgetItem("Résumé"));

afficherLivre();
remplirComboBox();
```

Graphiquement :

Création de la table Auteur :

```
/********************TABLEAU AUTEUR*************************/

//Création des lignes et colonnes
ui->tableWidget_Aut->setColumnCount(3);//Ajoute 3 colonne

/****************JE LA NOMME***************/
ui->tableWidget_Aut->setHorizontalHeaderItem(0,new QTableWidgetItem("N° auteur"));
ui->tableWidget_Aut->setHorizontalHeaderItem(1,new QTableWidgetItem("Nom de l'Auteur"));
ui->tableWidget_Aut->setHorizontalHeaderItem(2,new QTableWidgetItem("Select"));
```

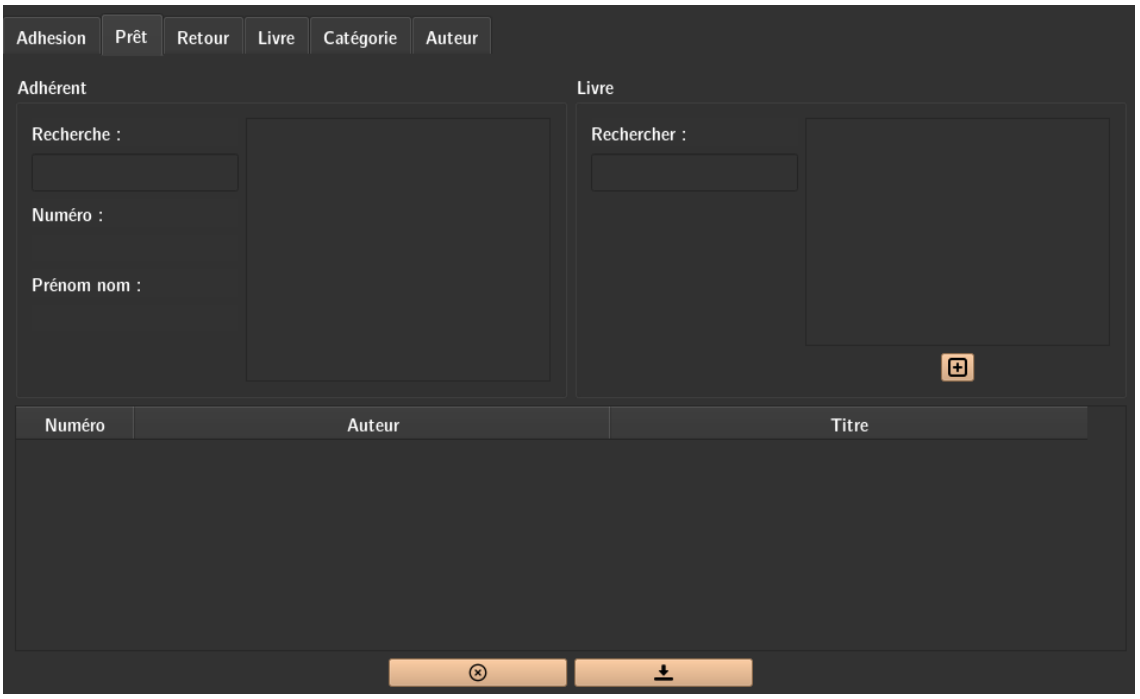
Graphiquement :



Création de tableau prêt :

```
/**********************TABLEAU PRÊT*************************/
ui->tableWidget_Pret->setColumnWidth(0,100);
ui->tableWidget_Pret->setColumnWidth(1,400);
ui->tableWidget_Pret->setColumnWidth(2,400);
```

Graphiquement :



Création de la fonction afficherLivre qui permet d’afficher les livres dans le tableau Livre :

```
void MainWindow::afficherLivre()
{
    qDebug()<<"void MainWindow::afficherLivre()";
    ui->tableWidget->setRowCount(0);
    QString txtRequeteLivre="SELECT numeroLivre, titreLivre, resumeLivre, ddpLivre, auteurLivre, categorieLivre, numeroCategorie, numeroAuteur FROM Livre;";//Requête pour afficher un livre
    QSqlQuery requeteLivre(txtRequeteLivre);//Conversion en Sql

    while(requeteLivre.next())//Boucle pour afficher en continue avec next (fetch)
    {
        noLigne=ui->tableWidget->rowCount();//Initialise les lignes

        ui->tableWidget->insertRow(noLigne);//Ajoute une ligne
        //Redimensionne les colonnes
        ui->tableWidget->setColumnWidth(0,30);
        ui->tableWidget->setColumnWidth(1,50);
        ui->tableWidget->setColumnWidth(2,200);
        ui->tableWidget->setColumnWidth(3,150);
        ui->tableWidget->setColumnWidth(4,130);
        ui->tableWidget->setColumnWidth(6,250);
        //Insère les valeurs dans les cellules correspondantes
        ui->tableWidget->setItem(noLigne,0,new QTableWidgetItem(requeteLivre.value("numeroLivre").toString()));
        ui->tableWidget->setCellWidget(noLigne,1,new QCheckBox(ui->tableWidget));//Ajout de la checkbox
        ui->tableWidget->setItem(noLigne,2,new QTableWidgetItem(requeteLivre.value("titreLivre").toString()));
        ui->tableWidget->setItem(noLigne,3,new QTableWidgetItem(requeteLivre.value("ddpLivre").toString()));
        ui->tableWidget->setItem(noLigne,4,new QTableWidgetItem(requeteLivre.value("auteurLivre").toString()));
        ui->tableWidget->setItem(noLigne,5,new QTableWidgetItem(requeteLivre.value("categorieLivre").toString()));
        ui->tableWidget->setItem(noLigne,6,new QTableWidgetItem(requeteLivre.value("resumeLivre").toString()));

    }
}
```

Création de la fonction remplirComboBox qui permet de remplir les combo Box de l'onglet livre avec des valeurs enregistrées dans la base :

```
void MainWindow::remplirComboBox()
{
    qDebug()<<"void MainWindow::remplirComboBox()";
    ui->comboBox_Auteur->clear();
    ui->comboBox_Auteur->addItem("...");
    ui->comboBox_Cat->clear();
    ui->comboBox_Cat->addItem("...");

    QString txtReqRemplirBoxA = "SELECT numeroAuteur, nomAuteur FROM Auteur;";
    QSqlQuery reqRemplirBoxA(txtReqRemplirBoxA);
    while(reqRemplirBoxA.next())
    {
        ui->comboBox_Auteur->addItem(reqRemplirBoxA.value("nomAuteur").toString(),reqRemplirBoxA.value("numeroAuteur").toString());
    }
    QString txtReqRemplirBoxC = "SELECT numeroCategorie, libelleCategorie FROM Categorie;";
    QSqlQuery reqRemplirBoxC(txtReqRemplirBoxC);
    while(reqRemplirBoxC.next())
    {
        ui->comboBox_Cat->addItem(reqRemplirBoxC.value("libelleCategorie").toString(),reqRemplirBoxC.value("numeroCategorie").toString());
    }
}
```

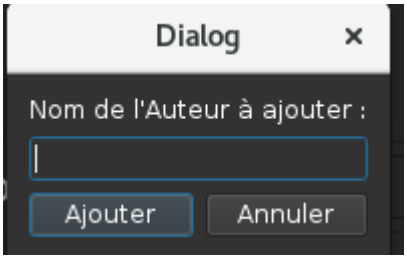
Graphiquement :

Voyons le code pour ces 2 bouton en pointillés qui sont très important :

```
void MainWindow::on_pushButton_Auteur_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Auteur_clicked()";
    DialogAuthor fenSaisieAuteur(this);
    if(fenSaisieAuteur.exec()==QDialog::Accepted)
    {
        //Création de la requête pour insérer l'auteur
        QString txtReqProchainIdAuteur="SELECT IFNULL((SELECT MAX(numeroAuteur)+1 FROM Auteur),10000)";
        QSqlQuery reqNumero(txtReqProchainIdAuteur);
        reqNumero.next();
        QString sonID=reqNumero.value(0).toString();
        QString txtReqInsertAuteur="INSERT INTO Auteur VALUES ("+sonID+",'"+fenSaisieAuteur.getNomAuteur()+"')";
        qDebug()<<txtReqInsertAuteur;
        QSqlQuery reqInsertAuteur(txtReqInsertAuteur);
        QString textAuteur=fenSaisieAuteur.getNomAuteur();
        ui->comboBox_Auteur->addItem(textAuteur,sonID);
        ui->comboBox_Auteur->setCurrentIndex(ui->comboBox_Auteur->findText(textAuteur));

        noLigne = ui->tableWidget_Aut->rowCount();//indique le nombre de ligne suivant "rowcount"
        qDebug()<<noLigne;
        ui->tableWidget_Aut->setRowCount(noLigne+1);//Ça l'ajoute

        ui->tableWidget_Aut->setItem(noLigne,0,new QTableWidgetItem(sonID));//Ça l'ajoute
        qDebug()<<sonID;

        //Insère la auteur
        ui->tableWidget_Aut->setItem(noLigne,1,new QTableWidgetItem(fenSaisieAuteur.getNomAuteur()));//Ça l'ajoute
        qDebug()<<fenSaisieAuteur.getNomAuteur();

        ui->tableWidget_Aut->setCellWidget(noLigne,2, new QCheckBox(ui->tableWidget_Aut));//Ajout de la checkbox
    }
}
```

```
void MainWindow::on_pushButton_Cat_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Cat_clicked()";
    DialogCategorie fenNouvelleCategorie(this);
    if(fenNouvelleCategorie.exec()==QDialog::Accepted)
    {
        //Création de la requête pour insérer la catégorie
        QString txtReqProchaineIdCategorie="SELECT IFNULL((SELECT MAX(numeroCategorie)+1 FROM Categorie),10000)";
        QSqlQuery reqNumero(txtReqProchaineIdCategorie);
        reqNumero.next();
        QString sonID=reqNumero.value(0).toString();
        QString txtReqInsertCategorie="INSERT INTO Categorie VALUES ("+sonID+",'"+fenNouvelleCategorie.getNomCategorie()+"')";
        qDebug()<<txtReqInsertCategorie;
        QSqlQuery reqInsertCategorie(txtReqInsertCategorie);
        QString textCat=fenNouvelleCategorie.getNomCategorie();
        ui->comboBox_Cat->addItem(textCat,sonID);
        ui->comboBox_Cat->setCurrentIndex(ui->comboBox_Cat->findText(textCat));

        noLigne = ui->tableWidget_Cat->rowCount();//indique le nombre de ligne suivant "rowcount"
        qDebug()<<noLigne;
        ui->tableWidget_Cat->setRowCount(noLigne+1);//Ça l'ajoute

        ui->tableWidget_Cat->setItem(noLigne,0,new QTableWidgetItem(sonID));//Ça l'ajoute
        qDebug()<<sonID;

        //Insère la auteur
        ui->tableWidget_Cat->setItem(noLigne,1,new QTableWidgetItem(fenNouvelleCategorie.getNomCategorie()));//Ça l'ajoute
        qDebug()<<fenNouvelleCategorie.getNomCategorie();

        ui->tableWidget_Cat->setCellWidget(noLigne,2, new QCheckBox(ui->tableWidget_Cat));//Ajout de la checkbox
    }
}
```

Au départ on ouvre un boite de dialog, affichant ceci :



C'est une fenêtre. Cette fenêtre permet, dans notre cas, d'ajouter soit un auteur, soit une catégorie. En cliquant sur ajouter, elle l'ajoute instantanément à la base de donnée et dans notre tableau.

Revenons à l'onglet Livre :

Création du slot clicked du bouton ajouter de l'onglet Livre :

```
void MainWindow::on_pushButton_Ajouter_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Ajouter_clicked()";
    QString numeroLignetxt="select ifnull((select max(numeroLivre)+1 from Livre),0);";//Affiche le nombre de livre +1 dans la bibliothèque
    QSqlQuery ajout(numeroLignetxt);//Convertit la requête en Sql
    ajout.next();//Commence la boucle
    QString numero=ajout.value(0).toString();//Initialise le numéro de livre

    noLigne = ui->tableWidget->rowCount();//indique le nombre de ligne suivant "rowcount"
    qDebug()<<noLigne;
    ui->tableWidget->setRowCount(noLigne+1);//Ça l'ajoute

    ui->tableWidget->setCellWidget(noLigne,1, new QCheckBox(ui->tableWidget));//Ajout de la checkbox

    QString Titre=ui->lineEdit_Titre->text();//Récupère la saisie de la case Titre
    Titre.replace("'","''").replace(";","");
    //Insère le Numéro
    ui->tableWidget->setItem(noLigne,2,new QTableWidgetItem(Titre));//Ça l'ajoute
    qDebug()<<Titre;

    QString DdP=ui->lineEdit_DdP->text();//Récupère la saisie de la case Date de publication
    DdP.replace("'","''").replace(";","");
    //Insère le DdP
    ui->tableWidget->setItem(noLigne,3,new QTableWidgetItem(DdP));//Ça l'ajoute
    qDebug()<<DdP;

    QString auteur=ui->comboBox_Auteur->currentText();//Récupère la saisie de la case Auteur
    auteur.replace("'","''").replace(";","");
    //Insère le auteur
    ui->tableWidget->setItem(noLigne,4,new QTableWidgetItem(auteur));//Ça l'ajoute
    qDebug()<<auteur;

    QString categorie=ui->comboBox_Cat->currentText();//Récupère la saisie de la case Catégorie
    categorie.replace("'","''").replace(";","");
    //Insère le categorie
    ui->tableWidget->setItem(noLigne,5,new QTableWidgetItem(categorie));//Ça l'ajoute
    qDebug()<<categorie;

    QString resume=ui->textEdit_Resume->toPlainText();//Récupère la saisie de la case Resumé
    resume=resume.replace("'","''").replace(";","");
    //Insère le resume
    ui->tableWidget->setItem(noLigne,6,new QTableWidgetItem(resume));//Ça l'ajoute
    qDebug()<<resume;
```

```
//Requete pour recuperer le numero de l'auteur
QString requeteRecupNumAuteur="SELECT numeroAuteur FROM Auteur WHERE nomAuteur='"+auteur+"';";
//Affiche ma requete dans le terminal
qDebug()<<requeteRecupNumAuteur;
//Execute la commande
QSqlQuery compterAuteur(requeteRecupNumAuteur);
compterAuteur.next();
//Recuperère la valeur dans une variable
QString numeroAuteur=compterAuteur.value("numeroAuteur").toString();
qDebug()<<numeroAuteur;

//Requete pour recuperer le numero categorie
QString requeteRecupNumCategorie="SELECT numeroCategorie FROM Categorie WHERE libelleCategorie='"+categorie+"';";
//Affiche ma requete dans le terminal
qDebug()<<requeteRecupNumCategorie;
//Execute la commande
QSqlQuery compterCat(requeteRecupNumCategorie);
compterCat.next();
//Recuperère la valeur dans une variable
QString numeroCategorie=compterCat.value("numeroCategorie").toString();
qDebug()<<numeroCategorie;
QString txtReqInsert = "INSERT INTO `Livre`(numeroLivre, titreLivre, ddpLivre, auteurLivre, categorieLivre, resumeLivre, numeroAuteur, numeroCategorie) VALUES
qDebug()<<txtReqInsert;
QSqlQuery reqInsert(txtReqInsert);//Convertit la requête en Sql
```

La suite de la requête pour insérer une catégorie :

QString txtReqInsert = "INSERT INTO `Livre`(numeroLivre, titreLivre, ddpLivre, auteurLivre, categorieLivre, resumeLivre, numeroAuteur, numeroCategorie) VALUES ("+numero+","'+Titre+"','+DdP+"','+auteur+"','+categorie+"','+resume+"','+numeroAuteur+","+numeroCategorie+");";

Donnant par exemple :
"INSERT INTO `Livre`(numeroLivre, titreLivre, ddpLivre, auteurLivre, categorieLivre, resumeLivre, numeroAuteur, numeroCategorie) VALUES "1","Les mots bleus”,”1965","Jean de la Fontaine","Fantaisie","Résumé du livre les mots bleus”,5,5);"

On a pu voir comment sécuriser les requête grâce au “replace” comme dans cette commande pour ajouter un résumé :
resume=resume.replace("'","''").replace(";","");

Création du slot clicked sur le bouton Modifier de l’onglet Livre:

```
void MainWindow::on_pushButton_Modifier_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Modifier_clicked()";
    if(!ui->tableWidget->selectedItems().empty())//Si la selection n'est pas vide
    {
        int ligneSelectionnee=ui->tableWidget->currentRow();//Sélectionne la ligne cliquée
        QString sonNumero=ui->tableWidget->item(ligneSelectionnee,0)->text();//Calcul la ligne actuelle

        //Change les infos dans le tableau et la base
        ui->tableWidget->item(ligneSelectionnee,2)->setText(ui->lineEdit_Titre->text());
        ui->tableWidget->item(ligneSelectionnee,3)->setText(ui->lineEdit_DdP->text());
        ui->tableWidget->item(ligneSelectionnee,4)->setText(ui->comboBox_Auteur->currentText());
        ui->tableWidget->item(ligneSelectionnee,5)->setText(ui->comboBox_Cat->currentText());
        ui->tableWidget->item(ligneSelectionnee,6)->setText(ui->textEdit_Resume->toPlainText());

        QString resume=ui->textEdit_Resume->toPlainText();
        resume=resume.replace("'","''").replace(";","");
        QString txtReqUpdate = "UPDATE Livre SET titreLivre='"+ui->lineEdit_Titre->text()+"', ddpLivre='"+ui->lineEdit_DdP->text()+"', auteurLivre='"+ui->comboBox_Auteur->currentText()+"', \
        categorieLivre='"+ui->comboBox_Cat->currentText()+"', resumeLivre='"+resume+"' where numeroLivre="+sonNumero;//Requête pour modifier un livre
        QSqlQuery reqUpdate(txtReqUpdate);//Convertit la requête en Sql
    }
}
```

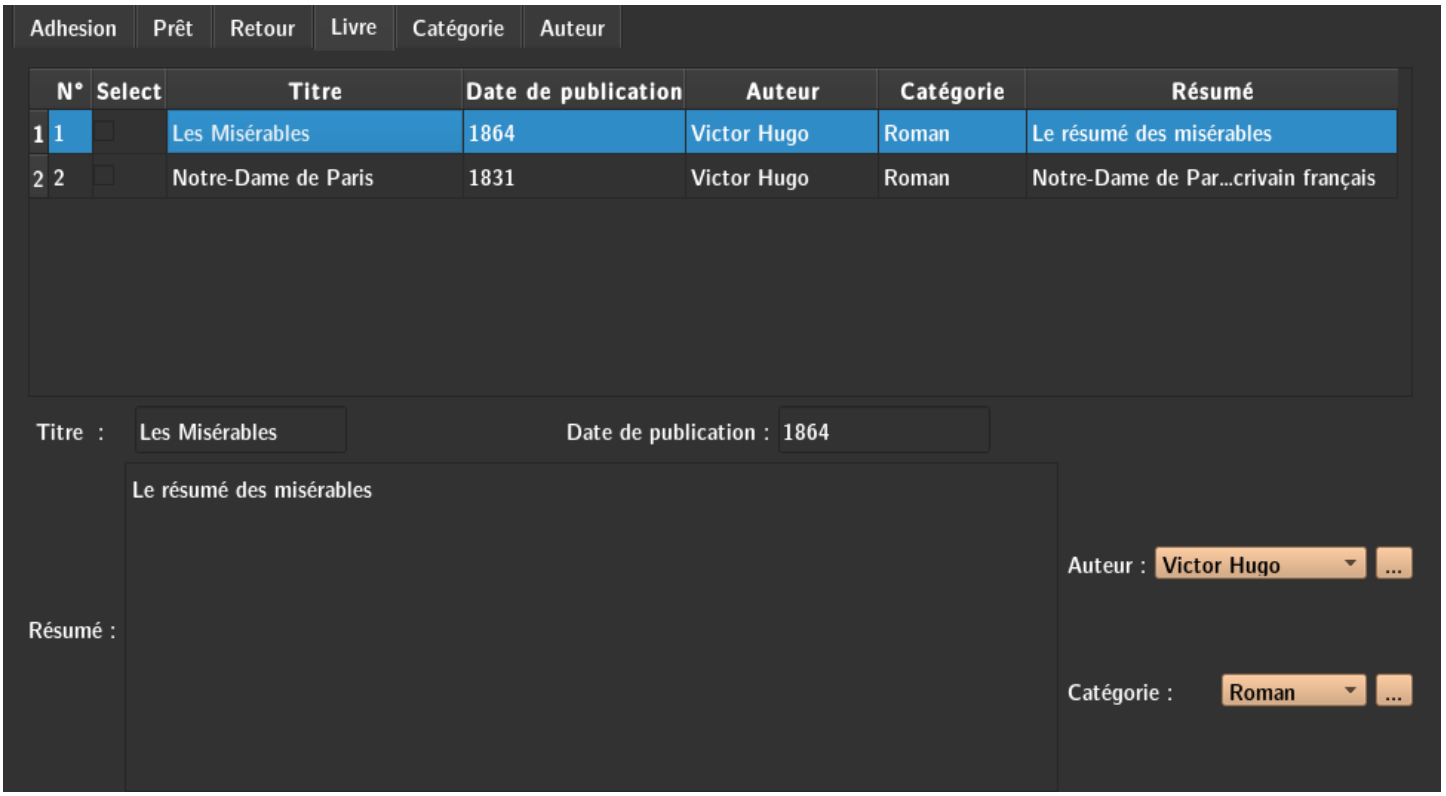
On retrouve ici notre “replace” pour notre résumé.

Exemple de requête pour modifier:
"UPDATE Livre SET titreLivre='Les Misérables', ddpLivre='1864', auteurLivre='Victor Hugo', categorieLivre='Roman', resumeLivre='Le résumé des misérables.' where numeroLivre=1"

Création du slot SelectionChanged sur notre tableau Livre. Je fais en sorte de remplir toutes les cases que l'utilisateur peut remplir, lui permettant de pouvoir les modifier sans tout réecrire.

```
void MainWindow::on_tableWidget_itemSelectionChanged()
{
    qDebug()<<"void MainWindow::on_tableWidget_itemSelectionChanged()";
    if(!ui->tableWidget->selectedItems().empty())//Si la selection n'est pas vide :
    {
        int noLigneSelectionne=ui->tableWidget->currentRow();//Calcul le numéro de la ligne séléctionnée
        ui->lineEdit_Titre->setText(ui->tableWidget->item(noLigneSelectionne,2)->text());//Insère les infos dans les lineEdits
        ui->lineEdit_DdP->setText(ui->tableWidget->item(noLigneSelectionne,3)->text());
        ui->comboBox_Auteur->setCurrentText(ui->tableWidget->item(noLigneSelectionne,4)->text());
        ui->comboBox_Cat->setCurrentText(ui->tableWidget->item(noLigneSelectionne,5)->text());
        ui->textEdit_Resume->setText(ui->tableWidget->item(noLigneSelectionne,6)->text());
    }
}
```

Comme ici, en cliquant sur l'oeuvre des misérables, la case titre, date de publication, résumé, et les comboBox auteur et catégorie ce sont remplis.n J'ai seulement a changer directement sur les cases que je souhaite sans avoir à tout refaire pour, possiblement, une faute de frappe par exemple.



Création du slot clicked sur le bouton supprimer :

```
void MainWindow::on_pushButton_Supprimer_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Supprimer_clicked()";
    for(int noLigne=ui->tableWidget->rowCount()-1; noLigne>=0;noLigne--)
    {
        if (((QCheckBox*)ui->tableWidget->cellWidget(noLigne,1))->isChecked())//Indique que c'est une checkbox, créer un cellwidget et vérifie s'il est checké
        {
            QString txtReqDel="DELETE FROM Livre WHERE numeroLivre="+ui->tableWidget->item(noLigne,0)->text();
            QSqlQuery reqDel(txtReqDel);
            qDebug()<<noLigne;
            qDebug()<<txtReqDel;
            ui->tableWidget->removeRow(noLigne);
        }
    }
}
```

On a pu apercevoir sur les screen ci-dessus, une colonne "select" avec un case à cocher. Ce slot permet de vérifier, en partant de la ligne la plus en bas, si cette case checkbox est cochée, si c'est le cas, lorsqu'on appuis sur supprimer, toute la ligne est supprimée visuellement et dans la base de donnée en éxécutant cette requête :
"DELETE FROM Livre WHERE numeroLivre=3"

TABLE CATEGORIE :

Création de la table Catégorie :

```
/*******************TABLEAU CATÉGORIE*************************/

//Création des lignes et colonnes
ui->tableWidget_Cat->setColumnCount(3);//Ajoute 3 colonne

/****************JE LA NOMME***************/
ui->tableWidget_Cat->setHorizontalHeaderItem(0,new QTableWidgetItem("N° catégorie"));
ui->tableWidget_Cat->setHorizontalHeaderItem(1,new QTableWidgetItem("Nom de la catégorie"));
ui->tableWidget_Cat->setHorizontalHeaderItem(2,new QTableWidgetItem("Select"));

afficherCategorie();
```

Avec la fonction pour afficher ce qu'il y a à l'intérieur:

```
void MainWindow::afficherCategorie()
{
    qDebug()<<"void MainWindow::afficherCategorie()";
    ui->tableWidget_Cat->setRowCount(0);
    QString txtReqCat="SELECT numeroCategorie, libelleCategorie FROM Categorie";
    QSqlQuery reqCat(txtReqCat);

    while(reqCat.next())
    {
        noLigne=ui->tableWidget_Cat->rowCount();

        ui->tableWidget_Cat->insertRow(noLigne);

        ui->tableWidget_Cat->setColumnWidth(0,100);
        ui->tableWidget_Cat->setColumnWidth(1,350);
        ui->tableWidget_Cat->setColumnWidth(2,50);

        ui->tableWidget_Cat->setItem(noLigne,0,new QTableWidgetItem(reqCat.value("numeroCategorie").toString()));
        ui->tableWidget_Cat->setItem(noLigne,1,new QTableWidgetItem(reqCat.value("libelleCategorie").toString()));
        ui->tableWidget_Cat->setCellWidget(noLigne,2,new QCheckBox(ui->tableWidget_Cat));

    }
}
```

Graphiquement :

Ici on créer le slot clicked du bouton ajouter pour, comme son nom l'indique, ajouter une catégorie :

```
void MainWindow::on_pushButton_Ajouter_Cat_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Ajouter_Cat_clicked()";
    QString numeroLignetxtCat="select ifnull((select max(numeroCategorie)+1 from Categorie),0);";//Affiche le nombre de catégorie +1 dans la bibliothèque
    QSqlQuery ajoutCat(numeroLignetxtCat);//Convertit la requête en Sql
    ajoutCat.next();//Commence la boucle
    QString numeroCat=ajoutCat.value(0).toString();//Initialise le numéro de la catégorie

    noLigne = ui->tableWidget_Cat->rowCount();//indique le nombre de ligne suivant "rowcount"
    qDebug()<<noLigne;
    ui->tableWidget_Cat->setRowCount(noLigne+1);//Ça l'ajoute


    ui->tableWidget_Cat->setItem(noLigne,0,new QTableWidgetItem(numeroCat));//Ça l'ajoute
    qDebug()<<numeroCat;

    QString Categorie=ui->lineEdit_Tab_Cat->text();//Récupère la saisie de la case Catégorie
    Categorie.replace("'","''").replace(";","");
    //Insère la catégorie
    ui->tableWidget_Cat->setItem(noLigne,1,new QTableWidgetItem(Categorie));//Ça l'ajoute
    qDebug()<<Categorie;

    ui->tableWidget_Cat->setCellWidget(noLigne,2, new QCheckBox(ui->tableWidget_Cat));//Ajout de la checkbox

    QString txtReqInsertCat = "INSERT INTO `Categorie`(numeroCategorie, libelleCategorie) VALUES ("+numeroCat+",'"+Categorie+"');";//Requête si l'on souhaite ajouter une catégorie
    qDebug()<<txtReqInsertCat;
    QSqlQuery reqInsertCat(txtReqInsertCat);//Convertit la requête en Sql
}
```

Le code est plus simple que sur la table livre, car il n'y a que trois colonnes à remplir, contre 7 pour la table LIvre.

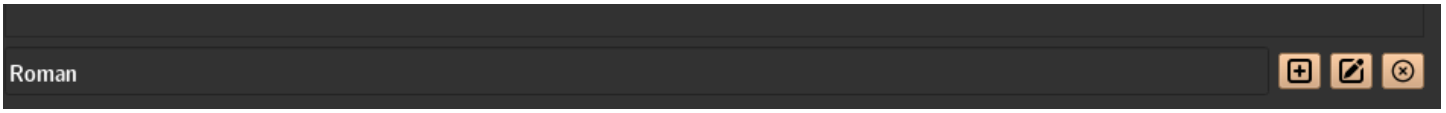
De même pour le bouton Modifier :

```
void MainWindow::on_pushButton_Modifier_Cat_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Modifier_Cat_clicked()";
    if(!ui->tableWidget_Cat->selectedItems().empty())//Si la selection n'est pas vide
    {
        int ligneSelectionnee=ui->tableWidget_Cat->currentRow();//Sélectionne la ligne cliquée
        QString sonNumero=ui->tableWidget_Cat->item(ligneSelectionnee,0)->text();//Calcul la ligne actuelle

        //Change les infos dans le tableau et la base
        ui->tableWidget_Cat->item(ligneSelectionnee, 1)->setText(ui->lineEdit_Tab_Cat->text());
        QString Categorie=ui->lineEdit_Tab_Cat->text();
        Categorie=Categorie.replace("'","''").replace(";","");
        QString txtReqUpdateCat = "UPDATE Categorie SET libelleCategorie='"+ui->lineEdit_Tab_Cat->text()+"' where Categorie.numeroCategorie = "+sonNumero+";";//Requête pour modifier une catégorie
        qDebug()<<txtReqUpdateCat;
        QSqlQuery reqUpdateCat(txtReqUpdateCat);//Convertit la requête en Sql
    }
}
```

On ajoute aussi la selectionChanged pour remplir les widgets :

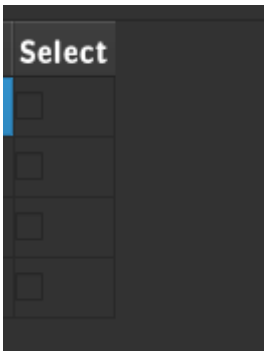
```
void MainWindow::on_tableWidget_Cat_itemSelectionChanged()
{
    qDebug()<<"void MainWindow::on_tableWidget_Cat_itemSelectionChanged()";
    if(!ui->tableWidget_Cat->selectedItems().empty())//Si la selection n'est pas vide :
    {
        int noLigneSelectionne=ui->tableWidget_Cat->currentRow();//Calcul le numéro de la ligne séléctionnée
        ui->lineEdit_Tab_Cat->setText(ui->tableWidget_Cat->item(noLigneSelectionne,1)->text());//Insère les infos dans les lineEdits
    }
}

void MainWindow::on_pushButton_Suppr_Cat_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Suppr_Cat_clicked()";
    for(int noLigne=ui->tableWidget_Cat->rowCount()-1; noLigne>=0;noLigne--)
    {
        if (((QCheckBox*)ui->tableWidget_Cat->cellWidget(noLigne,2))->isChecked())//Indique que c'est un checkbox, créer un cellwidget et vérifie s'il est checké
        {
            QString txtReqDelCat="DELETE FROM Categorie WHERE numeroCategorie="+ui->tableWidget_Cat->item(noLigne,0)->text();
            QSqlQuery reqDelCat(txtReqDelCat);
            qDebug()<<txtReqDelCat;
            ui->tableWidget_Cat->removeRow(noLigne);
        }
    }
}
```

Insérant notre sélection dans la case que l'utilisateur doit remplir :



On a également un bouton à cocher en cas de souhait de suppression, tout comme pour la table Livre.

```
void MainWindow::on_pushButton_Suppr_Cat_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Suppr_Cat_clicked()";
    for(int noLigne=ui->tableWidget_Cat->rowCount()-1; noLigne>=0;noLigne--)
    {
        if (((QCheckBox*)ui->tableWidget_Cat->cellWidget(noLigne,2))->isChecked())//Indique que c'est un checkbox, créer un cellwidget et vérifie s'il est checké
        {
            QString txtReqDelCat="DELETE FROM Categorie WHERE numeroCategorie="+ui->tableWidget_Cat->item(noLigne,0)->text();
            QSqlQuery reqDelCat(txtReqDelCat);
            qDebug()<<txtReqDelCat;
            ui->tableWidget_Cat->removeRow(noLigne);
        }
    }
}
```



Extrait de la requête envoyé lorsqu'on appuie sur supprimer pour la première catégorie:
"DELETE FROM Categorie WHERE numeroCategorie=1"

TABLE AUTEUR :

Pour la table Auteur, pas de grande différence avec catégorie.

Fonction pour afficher les Auteurs:

```
void MainWindow::afficherAuteur()
{
    qDebug()<<"void MainWindow::afficherAuteur()";
    ui->tableWidget_Aut->setRowCount(0);
    QString txtReqAut="SELECT numeroAuteur, nomAuteur FROM Auteur";
    QSqlQuery reqAut(txtReqAut);

    while(reqAut.next())
    {
        noLigne=ui->tableWidget_Aut->rowCount();

        ui->tableWidget_Aut->insertRow(noLigne);

        ui->tableWidget_Aut->setColumnWidth(0,100);
        ui->tableWidget_Aut->setColumnWidth(1,350);
        ui->tableWidget_Aut->setColumnWidth(2,50);

        ui->tableWidget_Aut->setItem(noLigne,0,new QTableWidgetItem(reqAut.value("numeroAuteur").toString()));
        ui->tableWidget_Aut->setItem(noLigne,1,new QTableWidgetItem(reqAut.value("nomAuteur").toString()));
        ui->tableWidget_Aut->setCellWidget(noLigne,2,new QCheckBox(ui->tableWidget_Aut));
    }
}
```

Fonction du slot clicked du bouton ajouter :

```
void MainWindow::on_pushButton_Ajouter_Aut_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Ajouter_Aut_clicked()";
    QString numeroLignetxtAut="select ifnull((select max(numeroAuteur)+1 from Auteur),0);";//Affiche le nombre d'auteur +1 dans la bibliothèque
    QSqlQuery ajoutAut(numeroLignetxtAut);//Convertit la requête en Sql
    ajoutAut.next();//Commence la boucle
    QString numeroAut=ajoutAut.value(0).toString();//Initialise le numéro de l'auteur

    noLigne = ui->tableWidget_Aut->rowCount();//indique le nombre de ligne suivant "rowcount"
    qDebug()<<noLigne;
    ui->tableWidget_Aut->setRowCount(noLigne+1);//Ça l'ajoute


    ui->tableWidget_Aut->setItem(noLigne,0,new QTableWidgetItem(numeroAut));//Ça l'ajoute
    qDebug()<<numeroAut;

    QString Auteur=ui->lineEdit_Tab_Aut->text();//Récupère la saisie de la case auteur
    Auteur.replace("'","''").replace(";","");
    //Insère la auteur
    ui->tableWidget_Aut->setItem(noLigne,1,new QTableWidgetItem(Auteur));//Ça l'ajoute
    qDebug()<<Auteur;

    ui->tableWidget_Aut->setCellWidget(noLigne,2, new QCheckBox(ui->tableWidget_Aut));//Ajout de la checkbox

    QString txtReqInsertAut = "INSERT INTO `Auteur`(numeroAuteur, nomAuteur) VALUES ("+numeroAut+",'"+Auteur+"');";//Requête si l'on souhaite ajouter un auteur
    qDebug()<<txtReqInsertAut;
    QSqlQuery reqInsertAut(txtReqInsertAut);//Convertit la requête en Sql
}
```

En parlant de ça, il faut toujours inséere l'id de notre type (catégorie, auteur, livre), c'est pour cela sue nous ajoutons cette requête : SELECT IFNULL((SELECT MAX(numeroAuteur)+1 FROM Auteur),0);" qui permet de le faire automatiquement.

Le slot clicked du bouton Modifier de la table Auteur :

```
void MainWindow::on_pushButton_Modifier_Aut_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Modifier_Aut_clicked()";
    if(!ui->tableWidget_Aut->selectedItems().empty())//Si la selection n'est pas vide
    {
        int ligneSelectionnee=ui->tableWidget_Aut->currentRow();//Sélectionne la ligne cliquée
        QString sonNumero=ui->tableWidget_Aut->item(ligneSelectionnee,0)->text();//Calcul la ligne actuelle

        //Change les infos dans le tableau et la base
        ui->tableWidget_Aut->item(ligneSelectionnee, 1)->setText(ui->lineEdit_Tab_Aut->text());
        QString Auteur=ui->lineEdit_Tab_Aut->text();
        Auteur=Auteur.replace("'","''").replace(";","");
        QString txtReqUpdateAut = "UPDATE Auteur SET nomAuteur='"+ui->lineEdit_Tab_Aut->text()+"' where Auteur.numeroAuteur = "+sonNumero+";";//Requête pour modifier un Auteur
        qDebug()<<txtReqUpdateAut;
        QSqlQuery reqUpdateAut(txtReqUpdateAut);//Convertit la requête en Sql
    }
}
```

La fonction selectionChanged, car la aussi il faut accompagner l'utilisateur, et lui faire perdre le moins de temps possible.

```
void MainWindow::on_tableWidget_Aut_itemSelectionChanged()
{
    qDebug()<<"void MainWindow::on_tableWidget_Aut_itemSelectionChanged()";
    if(!ui->tableWidget_Aut->selectedItems().empty())//Si la selection n'est pas vide :
    {
        int noLigneSelectionne=ui->tableWidget_Aut->currentRow();//Calcul le numéro de la ligne séléctionnée
        ui->lineEdit_Tab_Aut->setText(ui->tableWidget_Aut->item(noLigneSelectionne,1)->text());//Insère les infos dans les lineEdits
    }
}

void MainWindow::on_pushButton_Suppr_Aut_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Suppr_Aut_clicked()";
    for(int noLigne=ui->tableWidget_Aut->rowCount()-1; noLigne>=0;noLigne--)
    {
        if (((QCheckBox*)ui->tableWidget_Aut->cellWidget(noLigne,2))->isChecked())//Indique que c'est un checkbox, créer un cellwidget et vérifie s'il est checké
        {
            QString txtReqDelAut="DELETE FROM Auteur WHERE numeroAuteur="+ui->tableWidget_Aut->item(noLigne,0)->text();
            QSqlQuery reqDelAut(txtReqDelAut);
            qDebug()<<txtReqDelAut;
            ui->tableWidget_Aut->removeRow(noLigne);
        }
    }
}
```

Le slot clicked du bouton Ajouter de la table Auteur :

```
void MainWindow::on_pushButton_Suppr_Aut_clicked()
{
    qDebug()<<"void MainWindow::on_pushButton_Suppr_Aut_clicked()";
    for(int noLigne=ui->tableWidget_Aut->rowCount()-1; noLigne>=0;noLigne--)
    {
        if (((QCheckBox*)ui->tableWidget_Aut->cellWidget(noLigne,2))->isChecked())//Indique que c'est un checkbox, créer un cellwidget et vérifie s'il est checké
        {
            QString txtReqDelAut="DELETE FROM Auteur WHERE numeroAuteur="+ui->tableWidget_Aut->item(noLigne,0)->text();
            QSqlQuery reqDelAut(txtReqDelAut);
            qDebug()<<txtReqDelAut;
            ui->tableWidget_Aut->removeRow(noLigne);
        }
    }
}
```

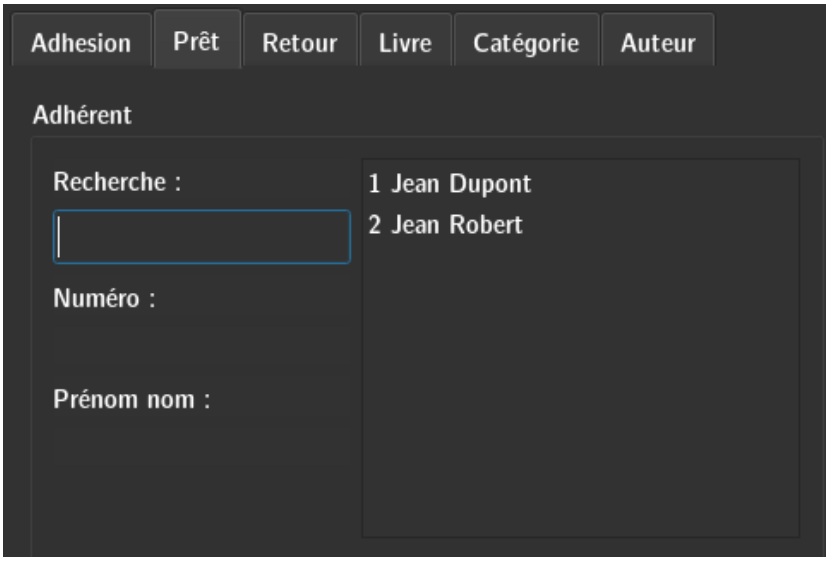
TABLEAU PRET :

Passons maintenant au tableau Pret :

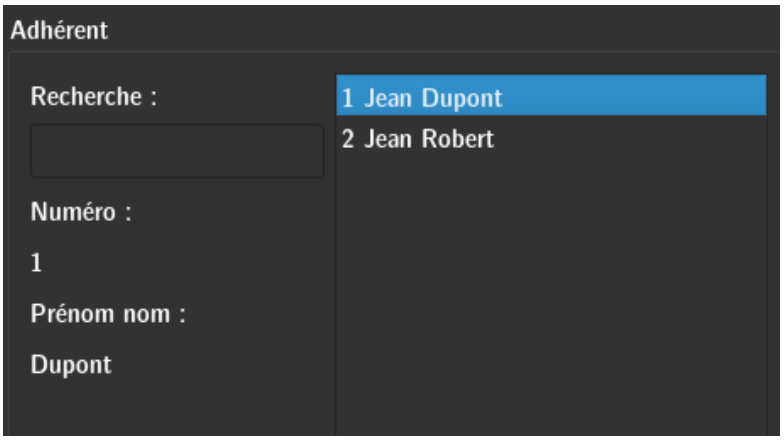
Tout d'abord, la fonction textEdited. Elle permet d'afficher dans une listWidget, ce que l'on recherche (ici l'adhérent).

```
void MainWindow::on_lineEditRechercheAdherentPret_textEdited(const QString &arg1)
{
    //Requete afficher numéro, prénom et nom de l'adhérent recherché
    QString txtReqAdherentNom="SELECT numeroClient, nomClient, prenomClient FROM Client WHERE numeroClient LIKE '%"+arg1+"%' OR nomClient LIKE '%"+arg1+"%' OR prenomClient LIKE '%"+arg1+"%'";
    QSqlQuery reqAdherentNom(txtReqAdherentNom);
    ui->listWidget_Adherent_Pret->clear();

    while (reqAdherentNom.next())
    {
        ui->listWidget_Adherent_Pret->addItem(reqAdherentNom.value("numeroClient").toString()+" "+reqAdherentNom.value("prenomClient").toString()+" "+reqAdherentNom.value("nomClient").toString());
        qDebug()<<txtReqAdherentNom;
    }
}
```

Graphiquement :



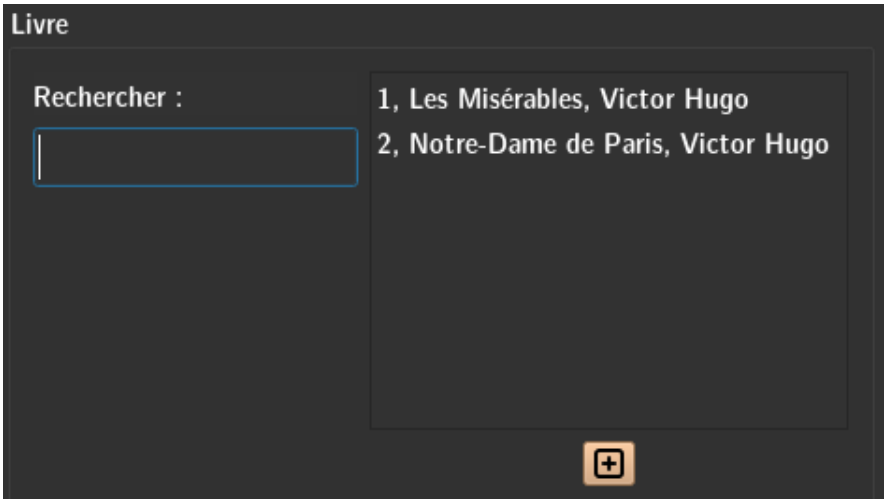
Petit bonus, en sélectionnant un adhérent, on affichés ces informations sous la barre de recherche :



Et pour un livre :

```
void MainWindow::on_lineEditRechercherLivrePret_textEdited(const QString &arg1)
{
    //Requete afficher id du livre, titre, auteur du livre recherché
    QString txtReqLivre="SELECT numeroLivre, titreLivre, auteurLivre FROM Livre WHERE numeroLivre LIKE '%"+arg1+"%' OR auteurLivre LIKE '%"+arg1+"%' OR titreLivre LIKE '%"+arg1+"%'";
    QSqlQuery reqLivre(txtReqLivre);
    ui->listWidget_Livre_Pret->clear();

    while (reqLivre.next())
    {
        ui->listWidget_Livre_Pret->addItem(reqLivre.value("numeroLivre").toString()+", "+reqLivre.value("titreLivre").toString()+", "+reqLivre.value("auteurLivre").toString());
        qDebug()<<txtReqLivre;
    }

}
```

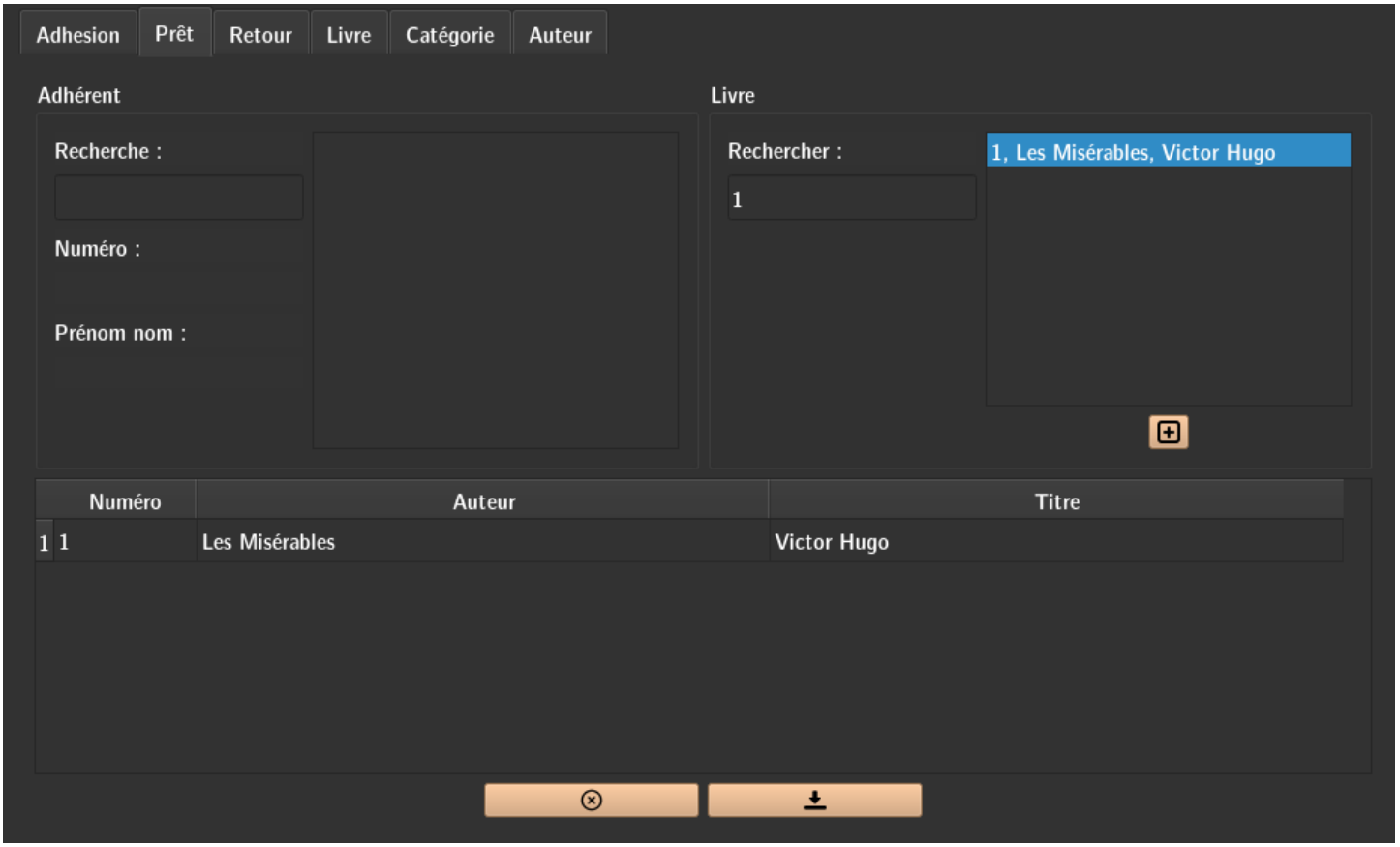
Graphiquement :



On peut apercevoir des virgule entre chaque type (numero, titre, auteur), cela permet de les séparer afin de les rentrer dans le tableau se situant dessous. Cette séparation est possible grâce à la méthode split(): listInfos=ui->listWidget_Livre_Pret->currentItem()->text().split(" ,")

On ajoute maintenant notre livre sélectionné par notre adhérent au tableau avant de confirmer :

```
void MainWindow::on_pushButtonAjouterLivrePret_clicked()
{
    if(!ui->listWidget_Livre_Pret->selectedItems().empty())//Si la selection n'est pas vide :
    {
        noLigne=ui->tableWidget_Pret->rowCount();//Initialise les lignes
        ui->tableWidget_Pret->setRowCount(noLigne+1);//Ajoute une ligne
        qDebug()<<noLigne;
        QStringList listInfosLivre;
        listInfosLivre=ui->listWidget_Livre_Pret->currentItem()->text().split(", ");//Sépare les infos lorsqu'il y a un espace

        ui->tableWidget_Pret->setItem(noLigne,0, new QTableWidgetItem(listInfosLivre[0]));//Insère les infos dans le label Numéro
        ui->tableWidget_Pret->setItem(noLigne,1, new QTableWidgetItem(listInfosLivre[1]));//Insère les infos dans le label Titre
        ui->tableWidget_Pret->setItem(noLigne,2, new QTableWidgetItem(listInfosLivre[2]));//Insère les infos dans le label Auteur
        qDebug()<<listInfosLivre;
    }
}
```



Si nous ajoutons un livre sans le vouloir, il suffit de sélectionner la ligne et de cliquer sur le bouton supprimer.

```
void MainWindow::on_pushButton_Supprimer_Livre_clicked()
{
    if (!ui->tableWidget_Pret->selectedItems().empty())//Indique que c'est un checkbox, créer un cellwidget et vérifie s'il est checké
        {
            ui->tableWidget_Pret->removeRow(noLigne);
            qDebug()<<noLigne;
        }
}
```

Fin intermédiaire :

Je ne suis pas allé plus loin par manque de temps.
Il faudrait ainsi créer la table Adhérent et retour, ainsi que faire fonctionner le prêt.